

Universidade do Minho
Mestrado em Engenharia Informártica

Course

Computing Systems Security (SSC)

Academic year 2026/2027

Block 1 Lab Report

Lab 1 — Environment Setup and Attack Surface · Lab 2 — Threat Modelling in Practice

Author

Marco António Ferreira Abreu — PG64003

Lecturer: Vítor Francisco Fonte

October 2026

Contents

Introduction **3**
Lab 1 — Environment Setup and First Contact: the Attack Surface of a Machine **4**
Preparation questions 4
P1. Screen capture and VM architecture 4
P2. What does it mean to enumerate a machine's attack surface, and why do it before any exploitation? 4
Part A — Orientation: what is this machine? 4
Exercise 1 — Identity and baseline 4
Part B — What listens? The network-facing surface 5
Exercise 2 — Listening sockets 5
Part C — What runs with privilege? Setuid binaries and sudo 9
Exercise 3 — Setuid binaries 9
Exercise 4 — Sudo rights 10
Part D — What acts on a schedule? Cron and writable paths 10
Exercise 5 — Scheduled jobs 10
Part E — What is exposed at rest? File permissions and secrets 11
Exercise 6 — Readable configuration 11
Part F — Consolidated attack surface inventory 12
Exercise 7 — Inventory 12
Lab 1 reflection 12
Lab 2 — Threat Modelling in Practice **14**
Preparation questions 14
P1. The four elements of a data-flow diagram 14
P2. What does STRIDE give you that an ad-hoc list of "problems" does not? 14
Part A — Attacker and assets 14
Exercise 1 — Attacker profile 14
Exercise 2 — Assets ranked by value to the attacker 15
Part B — Data-flow diagram of the machine 15
Exercise 3 — DFD of the VM 15
Exercise 3a — The boundary expected to be attacked most 17
Part C — STRIDE, element by element 17
Exercise 4 — STRIDE table 17
Exercise 4a — The three most serious cells 23
Exercise 4b — A threat found only by working through the grid 23
Part D — An attack tree 24
Exercise 5 — Goal G1: obtain root on the VM 24
Exercise 5a — Cheapest attack set 25
Exercise 5b — Deferred leaves 25
Part E — Prioritise, and measure 25
Exercise 6 — Ranked list of weaknesses 25
Exercise 6a — Defending the top item against the second 26
Exercise 6b — An attack surface metric 27

Exercise 6c — The policy line 28
Lab 2 reflection 28

Introduction

This report presents the work for Block 1 of the Computing Systems Security course at the University of Minho. Lab 1 enumerates the attack surface of the course virtual machine (SSC lab base image, version 1.0, Ubuntu 24.04 LTS, amd64, hostname ssc-lab), and Lab 2 turns that inventory into a threat model of the same machine. The model is specific to this VM: it rests on report.service on port 8081, the setuid binary /usr/local/bin/backup-tool, the cron job /etc/cron.d/ssc-maint that runs /opt/maint/rotate.sh as root, the credentials file /etc/report/report.conf and the sudo rule of the svc-report account.

In line with the course method, nothing was exploited: every observation comes from read-only commands and, where the underlying mechanism has not yet been taught, the observation is recorded and the judgement deferred. The credentials that appear in the captures are fake values planted by the course lab image for educational purposes; they are not real secrets.

Lab 1 — Environment Setup and First Contact: the Attack Surface of a Machine

Preparation questions

P1. Screen capture and VM architecture

Before the session, date; cat /etc/ssc-image-release was run. The capture below, dated Saturday 3 October 2026, 16:21:34 WEST, is the proof that the machine was running before the lab:

```
student@ssc-lab:~/ssc-lab1$ date
Sat  3 Oct 16:21:34 WEST 2026
student@ssc-lab:~/ssc-lab1$ cat /etc/ssc-image-release
SSC lab base image
version: 1.0
built: cloud-init single-source, dual-arch native
arch: amd64
seeds ran:  seed-week1-surface.sh seed-lab6-setuid-sudo.sh
seed gate: asserted
cgroup v2 OK, services active
```

Figure 1 – Output of date and cat /etc/ssc-image-release on the ssc-lab VM.

The VM architecture is amd64 (field arch: amd64 in /etc/ssc-image-release).

P2. What does it mean to enumerate a machine's attack surface, and why do it before any exploitation?

Enumerating a machine's attack surface means systematically observing and recording every point where an adversary can interact with it. It is done before any exploitation because only by knowing what exists can one decide what deserves to be evaluated; otherwise exploitation would be random, could be incomplete, and the threat model would have no foundation.

Part A — Orientation: what is this machine?

Exercise 1 — Identity and baseline

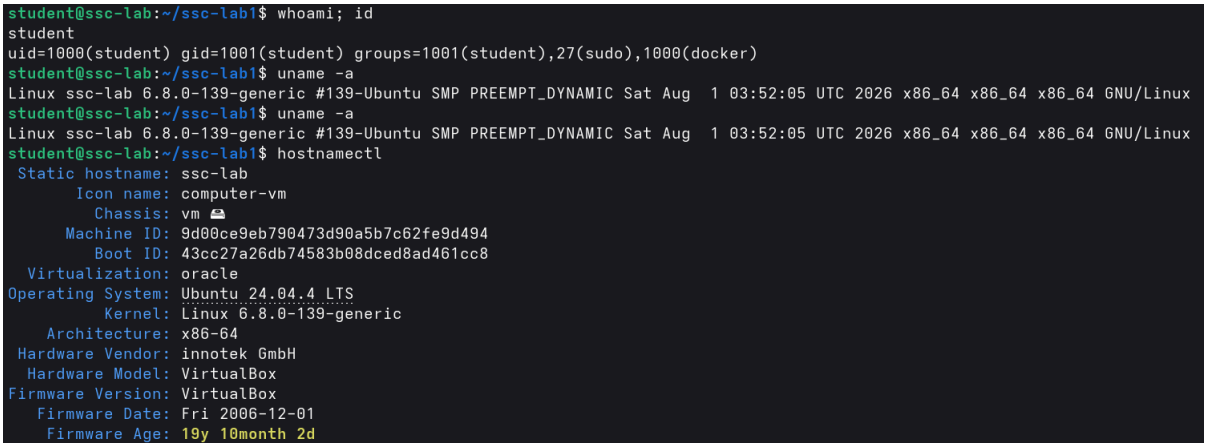


Figure 2 – Identity and baseline commands (whoami; id, uname -a, /etc/os-release, hostnamectl).

1a. The student account belongs to the groups student, sudo and docker. The most security-relevant one is docker.

1b. “Shared kernel” suggests that the boundary between two containers is weak, since the isolation of every container depends on a single kernel. If an adversary exploits a kernel flaw that allows a container escape, they can affect the host or the other containers.

1c. The hypervisor keeps the host and the guest apart, so that if the guest is compromised the host is not affected. A shared folder would be an exception, because it would create a bridge between the two; since this VM runs exploits, something malicious could reach the host through it, which is why the course forbids it.

Part B — What listens? The network-facing surface

Exercise 2 — Listening sockets

2a. There are four TCP ports in the LISTEN state: 8081, 53, 22 and 46735.

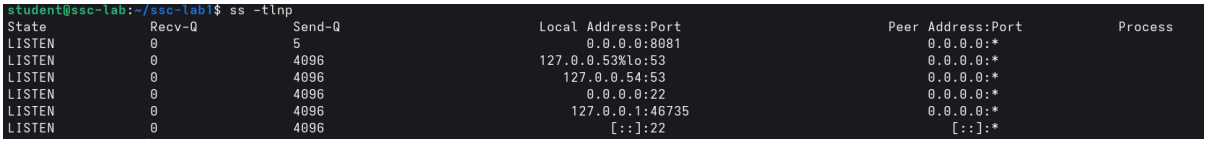


Figure 3 – ss -tlnp.

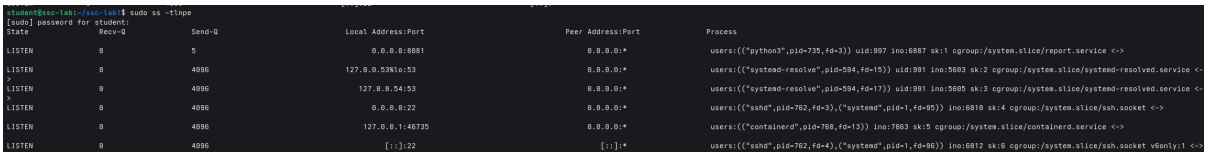


Figure 4 – sudo ss -tlnpe (extended information, including the owning uid).

```
student@ssc-lab:~/ssc-lab1$ ps -o user,pid,cmd -p 735

USER         PID CMD
svc-rep+     735 /usr/bin/python3 /opt/report/server.py
student@ssc-lab:~/ssc-lab1$ ps -o user,pid,cmd -p 594

USER         PID CMD
systemd+     594 /usr/lib/systemd/systemd-resolved
student@ssc-lab:~/ssc-lab1$ ps -o user,pid,cmd -p 762

USER         PID CMD
root         762 sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups
student@ssc-lab:~/ssc-lab1$ ps -o user,pid,cmd -p 768
USER         PID CMD
root         768 /usr/bin/containerd
```

Figure 5 – ps -o user,pid,cmd for each listening PID.

- Port 8081 is bound to 0.0.0.0; the program is python3 running /opt/report/server.py, as the user svc-report.
- Port 53 is bound to 127.0.0.53%lo and 127.0.0.54; the program is /usr/lib/systemd/systemd-resolved, as the systemd resolver user (shown truncated as systemd+).
- Port 22 is bound to 0.0.0.0 and to all IPv6 interfaces ([::]); the program is /usr/sbin/sshd, as root.
- Port 46735 is bound to 127.0.0.1; the program is /usr/bin/containerd, as root.

Binding to 127.0.0.1 means binding to the loopback interface: the port only accepts connections from the machine itself. Binding to 0.0.0.0 means binding to every IPv4 interface: the port accepts connections both from the machine and from the network. Ports 22 and 8081 are therefore reachable from the network, while ports 46735 and 53 are only reachable locally.

2b. The service listening on port 8081 is python3 (PID 735), running /opt/report/server.py as the user svc-report, so it does not run as root. Its unit file is /etc/systemd/system/report.service.

```
student@ssc-lab:~/ssc-lab1$ sudo ss -tlnp | grep 8081
[sudo] password for student:
LISTEN 0      5            0.0.0.0:8081       0.0.0.0:*    users:(("python3",pid=735,fd=3))
student@ssc-lab:~/ssc-lab1$ systemctl status report.service
● report.service - Status report service (SSC Week 1 seeded surface)
     Loaded: loaded (/etc/systemd/system/report.service; enabled; preset: enabled)
     Active: active (running) since Sat 2026-10-03 16:01:16 WEST; 24h ago
   Main PID: 735 (python3)
      Tasks: 1 (limit: 4653)
     Memory: 10.4M (peak: 10.6M)
        CPU: 2.037s
     CGroup: /system.slice/report.service
             └─735 /usr/bin/python3 /opt/report/server.py

Warning: some journal files were not opened due to insufficient permissions.
student@ssc-lab:~/ssc-lab1$ systemctl cat report.service
# /etc/systemd/system/report.service
[Unit]
Description=Status report service (SSC Week 1 seeded surface)
After=network.target
[Service]
User=svc-report
ExecStart=/usr/bin/python3 /opt/report/server.py
Restart=on-failure
[Install]
WantedBy=multi-user.target
```

Figure 6 – sudo ss -tlnp | grep 8081, systemctl status report.service and systemctl cat report.service.

Running the service as a low-privilege user is a defensive choice: if a vulnerability is found in the service and an attacker compromises it, they only obtain that user's permissions, not full control of the machine, which limits the damage.

2c. curl sends an HTTP request to a server and prints the response. Without any authentication, the response reveals the database user name, report. Because port 8081 is bound to 0.0.0.0, it accepts connections on every interface, so any client that reaches it obtains this information. Note for later: where are this account's credentials stored on the system?

```
student@ssc-lab:~/ssc-lab1$ curl -s http://127.0.0.1:8081/ | head -40
report service ok; db_user=report
```

Figure 7 – curl -s http://127.0.0.1:8081/ | head -40.

2d. Prediction, before asking the server: the value in 50-cloud-init.conf would win, because that initialisation file would decide whether password authentication is allowed. (The reasoning behind this prediction turned out to be wrong; see 2d-i.)

```
student@ssc-lab:~/ssc-lab1$ ls /etc/ssh/sshd_config.d/
50-cloud-init.conf  60-cloudimg-settings.conf
student@ssc-lab:~/ssc-lab1$ sudo grep -r PasswordAuthentication /etc/ssh/sshd_config.d/ /etc/ssh/sshd_config
/etc/ssh/sshd_config.d/60-cloudimg-settings.conf:PasswordAuthentication no
/etc/ssh/sshd_config.d/50-cloud-init.conf:PasswordAuthentication yes
/etc/ssh/sshd_config:#PasswordAuthentication yes
/etc/ssh/sshd_config:# PasswordAuthentication.  Depending on your PAM configuration,
/etc/ssh/sshd_config:# PAM authentication, then enable this but set PasswordAuthentication
student@ssc-lab:~/ssc-lab1$ grep -n '^Include' /etc/ssh/sshd_config
12:Include /etc/ssh/sshd_config.d/*.conf
```

Figure 8 – Drop-ins in /etc/ssh/sshd_config.d/, PasswordAuthentication occurrences and the Include line of sshd_config.

2d-i. sshd -T reports passwordauthentication yes. The prediction was right, but its justification was not. The Include near the top of sshd_config reads the drop-ins in lexical filename order and, for each keyword, sshd keeps the first value it sees and discards the later ones. It therefore takes PasswordAuthentication yes from 50-cloud-init.conf, which makes 60-cloudimg-settings.conf dead weight for this keyword.

```
student@ssc-lab:~/ssc-lab1$ sudo sshd -T | grep -iE '^passwordauthentication|^pubkeyauthentication|^permitrootlogin|^authenticationmethods|^truste
dusercakeys|^authorizedkeysfile'
[sudo] password for student:
permitrootlogin without-password
pubkeyauthentication yes
passwordauthentication yes
trustedusercakeys none
authorizedkeysfile .ssh/authorized_keys .ssh/authorized_keys2
authenticationmethods any
```

Figure 9 – Effective configuration reported by sudo sshd -T for the six keywords.

2d-i-bis. Of the six keywords, PasswordAuthentication is the only one actually set in a file; the others only appear in comments. The values that are not written in any file — permitrootlogin without-password, pubkeyauthentication yes, trustedusercakeys none, authorizedkeysfile .ssh/authorized_keys .ssh/authorized_keys2 and authenticationmethods any — were supplied by the daemon itself as defaults. The value any for authenticationmethods admits any authentication method, for example password authentication, which AuthenticationMethods publickey would not admit, since it requires key-based authentication.

```
student@ssc-lab:~/ssc-lab1$ sudo grep -ri passwordauthentication /etc/ssh/
/etc/ssh/sshd_config.d/60-cloudimg-settings.conf:PasswordAuthentication no
/etc/ssh/sshd_config.d/50-cloud-init.conf:PasswordAuthentication yes
/etc/ssh/sshd_config:#PasswordAuthentication yes
/etc/ssh/sshd_config:# PasswordAuthentication.  Depending on your PAM configuration,
/etc/ssh/sshd_config:# PAM authentication, then enable this but set PasswordAuthentication
/etc/ssh/ssh_config:#   PasswordAuthentication yes
student@ssc-lab:~/ssc-lab1$ sudo grep -ri pubkeyauthentication /etc/ssh/
/etc/ssh/sshd_config:#PubkeyAuthentication yes
student@ssc-lab:~/ssc-lab1$ sudo grep -ri permitrootlogin /etc/ssh/
/etc/ssh/sshd_config:#PermitRootLogin prohibit-password
/etc/ssh/sshd_config:# the setting of "PermitRootLogin prohibit-password".
student@ssc-lab:~/ssc-lab1$ sudo grep -ri authenticationmethods /etc/ssh/
student@ssc-lab:~/ssc-lab1$ sudo grep -ri trustedusercakeys /etc/ssh/
student@ssc-lab:~/ssc-lab1$ sudo grep -ri authorizedkeysfile /etc/ssh/
/etc/ssh/sshd_config:#AuthorizedKeysFile        .ssh/authorized_keys .ssh/authorized_keys2
```

Figure 10 – grep -ri for each keyword under /etc/ssh/.

2d-ii. When auditing a service whose configuration is assembled from fragments, reading a single file is not enough: that file can give a wrong idea of the real posture, because the effective value of each keyword depends on the Include order (sshd reads the files in lexical order and keeps the first value of each keyword), and keywords that are not set anywhere take the daemon's default. To know the real behaviour, one must ask the service itself for its effective configuration (sshd -T).

Part C — What runs with privilege? Setuid binaries and sudo

Exercise 3 — Setuid binaries

3a. The non-standard entry is /usr/local/bin/backup-tool; the other 13 are standard Ubuntu entries. The full list of 14 entries is in the capture below.

```
student@ssc-lab:~/ssc-lab1$ find / -perm -4000 -type f 2>/dev/null
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/usr/lib/openssh/ssh-keysign
/usr/lib/polkit-1/polkit-agent-helper-1
/usr/bin/fusermount3
/usr/bin/mount
/usr/bin/sudo
/usr/bin/newgrp
/usr/bin/su
/usr/bin/umount
/usr/bin/chsh
/usr/bin/passwd
/usr/bin/chfn
/usr/bin/gpasswd
/usr/local/bin/backup-tool
```

Figure 11 – find / -perm -4000 -type f.

3b. The file is owned by root and its permissions are -rwsr-xr-x: the owner can read, write and execute, and the s in place of the owner's x is the setuid bit; group and others can read and execute. The file is 141848 bytes. The --version option prints cp (GNU coreutils) 9.4, so backup-tool is a copy of cp with the setuid bit set: whoever runs it gets a process running with the privileges of the file's owner, root.

```
student@ssc-lab:~/ssc-lab1$ ls -l /usr/local/bin/backup-tool
-rwsr-xr-x 1 root root 141848 Sep  4 09:01 /usr/local/bin/backup-tool
student@ssc-lab:~/ssc-lab1$ /usr/local/bin/backup-tool --version 2>&1 | head -1
cp (GNU coreutils) 9.4
student@ssc-lab:~/ssc-lab1$ /usr/local/bin/backup-tool 2>&1 | head
/usr/local/bin/backup-tool: missing file operand
Try '/usr/local/bin/backup-tool --help' for more information.
```

Figure 12 – ls -l, --version and a run without arguments of /usr/local/bin/backup-tool.

Open question: to what extent can an unprivileged user read or write, through backup-tool, files they could not read or write on their own, given that it is cp running as root?

Exercise 4 — Sudo rights

4a. sudo -l shows that student, through the rule (ALL : ALL) ALL, can run any command as any user and any group; the second rule, (ALL) ALL, likewise allows any command as any user. student is therefore a full administrator of the machine, with the same privileges as root. This is expected, since it is the owner's own VM.

```
student@ssc-lab:~/ssc-lab1$ sudo -l
[sudo] password for student:
Matching Defaults entries for student on ssc-lab:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin, use_pty

User student may run the following commands on ssc-lab:
    (ALL : ALL) ALL
    (ALL) ALL
```

Figure 13 – sudo -l for the student account.

Part D — What acts on a schedule? Cron and writable paths

Exercise 5 — Scheduled jobs

5a. The job that runs as root is /etc/cron.d/ssc-maint. It runs every 10 minutes (*/10 * * * *), as the user root, and executes /opt/maint/rotate.sh. Other jobs also run as root (/etc/cron.d/e2scrub_all and /etc/cron.d/sysstat), but they are standard on Ubuntu.

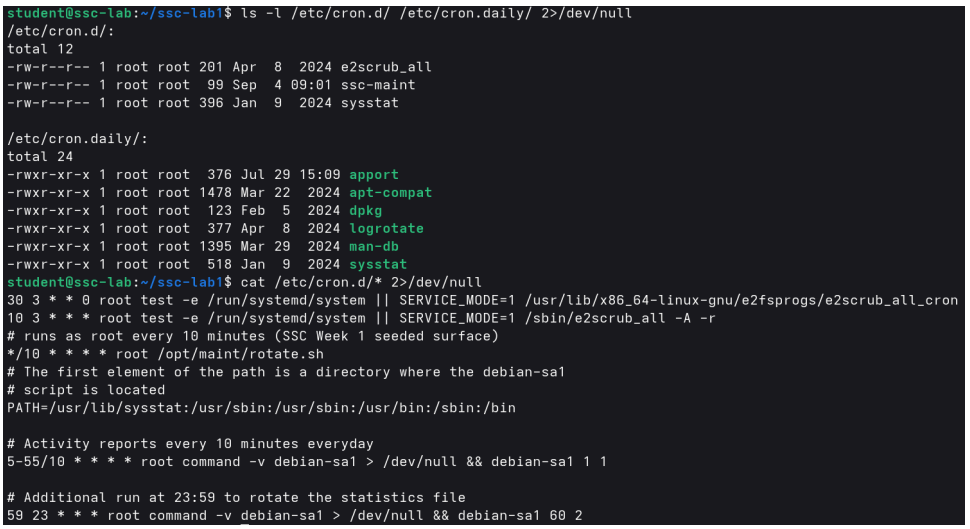


Figure 14 – ls -l and cat of /etc/cron.d/.

5b. The permission string -rwxrwxrwx shows that, besides root, any user can read, write and execute this script. Since cron runs it as root every 10 minutes and any user can change it, this creates a privilege-escalation problem: lower-privileged users can change code that will be executed with higher privileges, in this case root.

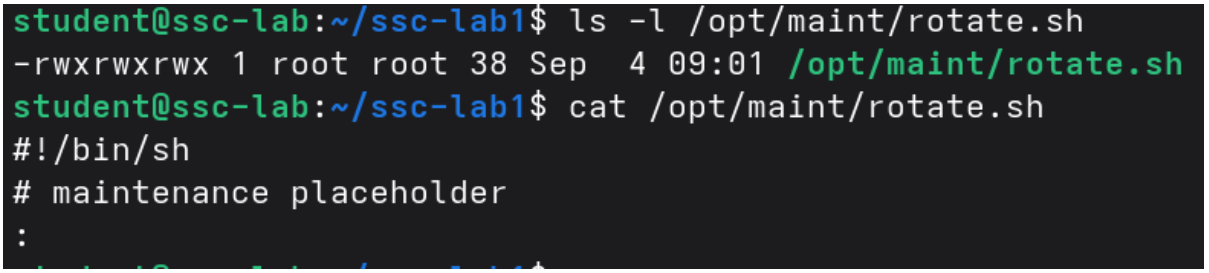


Figure 15 – ls -l and cat of /opt/maint/rotate.sh.

Part E — What is exposed at rest? File permissions and secrets

Exercise 6 — Readable configuration

6a. The permissions of /etc/report/report.conf are -rw-r--r--: any user can read the file, and only its owner, root, can write it. The file contains credentials.

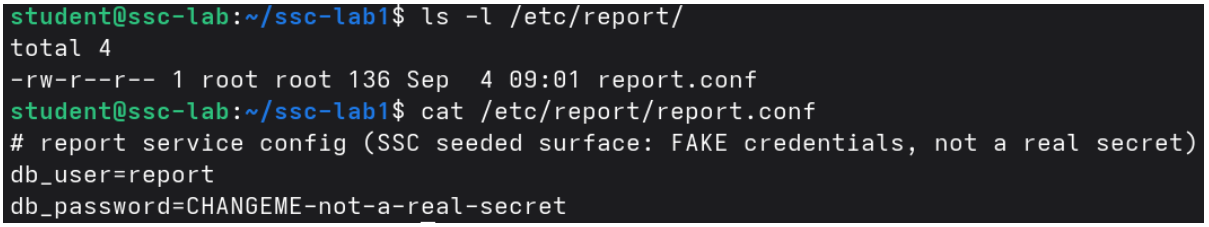


Figure 16 – ls -l /etc/report/ and cat /etc/report/report.conf. Note: the credentials shown are deliberately fake values planted by the course lab image for educational purposes (the file itself says so); they are not real secrets.

6b. If only root can read the file, an attacker has to escalate privileges before obtaining the credentials, which adds another barrier. If every user can read it, compromising any single account is enough for the attacker to use the credential.

Part F — Consolidated attack surface inventory

Exercise 7 — Inventory

#	Surface item	Dimension	Runs/owned as	Who can reach/modify it	Open question
1	report service (python3 /opt/report/server.py, port 8081)	What listens	svc-report	Any client that can reach port 8081, without authentication.	What information or actions does this service expose, and what can whoever compromises it do?
2	/usr/local/bin/backup-tool	What is privileged	root	Any user can run it.	To what extent can an unprivileged user read or write, through it, files they could not access on their own?
3	/etc/cron.d/ssc-maint (script: /opt/maint/rotate.sh)	What is scheduled	root	Any user can modify the script.	What mechanisms allow an unprivileged user to take advantage of a script that runs as root?
4	/etc/report/report.conf	What is at rest	root	Any user can read this configuration; only root can write it.	Which database does this credential give access to, and what does it allow?

Table 1 – Attack surface inventory of the VM.

Lab 1 reflection

Thinking in dimensions is useful because each dimension has its own question and its own observation tool, which makes enumeration systematic: I know where I have already looked and where I still need to look. In a single list, problems from different dimensions get mixed, and one may jump from a problem in one dimension to another without ever being sure that each dimension has been properly enumerated. For example, rotate.sh only appeared when I looked at the “what is scheduled” dimension; in a single list of problems it would have stayed hidden behind the network-facing services, which are the first to be seen.

Making a single observation, jumping straight to a conclusion and assuming that it is a vulnerability can lead to a wrong conclusion. Moreover, jumping to hasty conclusions about vulnerabilities can cost a practitioner their credibility. For example,

backup-tool has the setuid bit and therefore runs with the privileges of the file’s owner, root. Its real reach is not yet known, so it was only enumerated.

Lab 2 — Threat Modelling in Practice

Preparation questions

P1. The four elements of a data-flow diagram

The four elements of a DFD are external entities, processes, data stores and data flows. Trust boundaries are drawn over the diagram, but they are not one of the four elements.

P2. What does STRIDE give you that an ad-hoc list of “problems” does not?

STRIDE applies the same six threat categories to every DFD element, which gives systematic coverage and reduces the predictable omissions of an ad-hoc list (which only includes the attacks we already know), although it does not guarantee that the model is complete.

Part A — Attacker and assets

Exercise 1 — Attacker profile

The VM owner, the student account, is a full administrator ((ALL : ALL) ALL in sudo -l), which is expected and out of scope. The modelled attacker already has a foothold: it runs code as the service account svc-report, the identity behind the port-8081 service. To find out what this account can do with sudo, sudo -l -U svc-report was run from the student account:

```
student@ssc-lab:~/ssc-lab2$ sudo -l -U svc-report
[sudo] password for student:
Matching Defaults entries for svc-report on ssc-lab:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin, use_pty

User svc-report may run the following commands on ssc-lab:
    (ALL) NOPASSWD: /usr/bin/find
```

Figure 17 – sudo privileges of svc-report: (ALL) NOPASSWD: /usr/bin/find.

Open question: svc-report can run /usr/bin/find as any user, without a password (NOPASSWD). What can this find actually reach as root on the machine? This capability is recorded in the model as permitted, but with unmeasured reach.

Position: the attacker runs code as svc-report, an account without root privileges that runs report.service on port 8081. It is not the machine’s owner, so it does not have the rights student has through sudo. From this position it can already read report.conf, both because this is the identity the service uses to read it and because any user can read it. It can also run /usr/local/bin/backup-tool, a cp with the setuid bit

that any user can run, and read and write /opt/maint/rotate.sh, since any user can read, write and execute that file. Finally, it can run /usr/bin/find via sudo as any user, including root, without a password — a permitted capability whose reach has not yet been measured.

The attacker reaches the machine through the service on port 8081, which is bound to 0.0.0.0 and answers without authentication. Reachability from another machine is an assumption, since in Lab 1 the service was only reached from 127.0.0.1.

Goal: the attacker wants root on the machine in order to persist on it. As a secondary goal, it wants to take the credentials stored in report.conf.

Motivation: an exposed service without authentication is a cheap way in. Once inside, the machine gives the attacker access to database credentials that can be reused on other systems, and serves as a foothold to move further into the network.

Exercise 2 — Assets ranked by value to the attacker

1. Code execution as root (/opt/maint/rotate.sh, run by cron as root): whoever executes code as root controls the whole machine and reaches every other asset.
2. Sudo policy (the rules shown by sudo -l, including the NOPASSWD rule of svc-report): whoever changes it decides who can become root, which amounts to persistent root.
3. Integrity of the files protected against backup-tool: a cp running as root can, in principle, read or write files the attacker could not access, so what is at stake is everything only root can read or write. Its real reach has not yet been measured.
4. Database credentials (/etc/report/report.conf): the only asset with value outside the machine, because they can be reused on other systems. For this attacker, however, confidentiality is already lost, since svc-report reads them.
5. Availability of the report service: of little value to this attacker, because it is its own entry point. For the operator it remains an asset.

Part B — Data-flow diagram of the machine

Exercise 3 — DFD of the VM

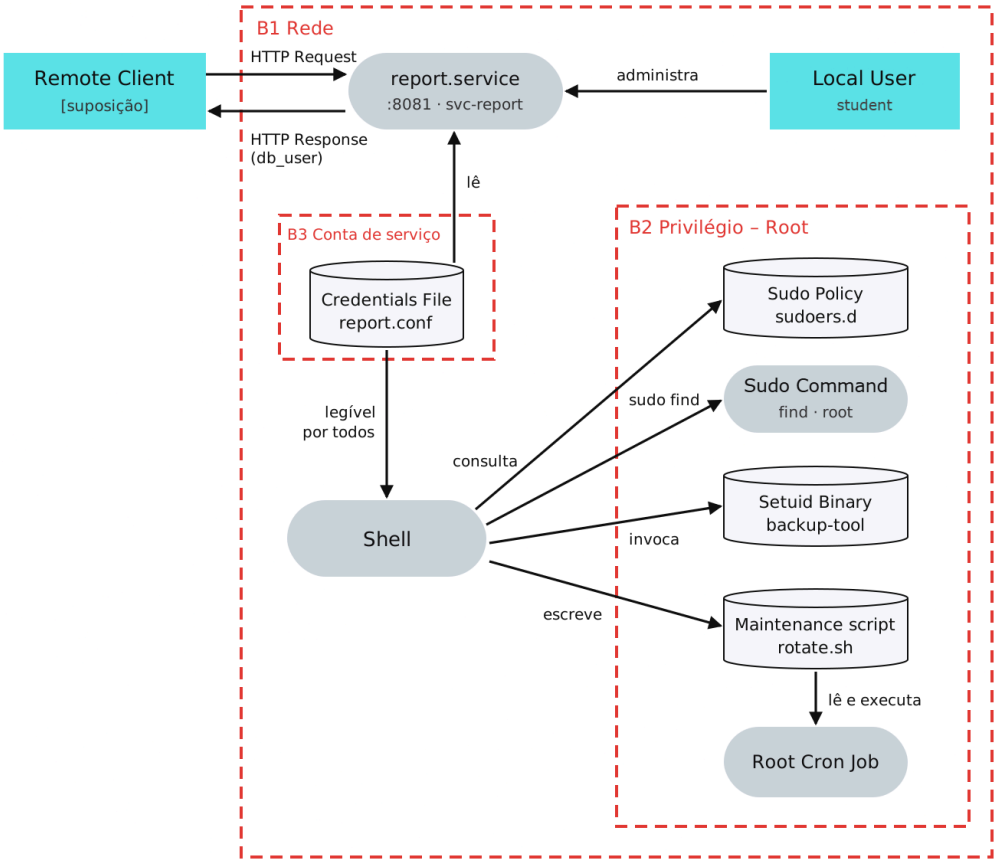


Figure 18 – DFD of the lab VM with trust boundaries B1 (Network), B2 (Privilege – Root) and B3 (Service account); the Remote Client is marked as an assumption. Labels in Portuguese: Rede = Network, Privilégio = Privilege, Conta de serviço = Service account, suposição = assumption, administra = administers, lê = reads, legível por todos = world-readable, consulta = queries, invoca = invokes, escreve = writes, lê e executa = reads and executes.

The diagram starts from the Lab 1 inventory. External entities: the Remote Client, marked as an assumption because in Lab 1 the service was only reached from the VM itself, and the Local User (student). Processes: report.service (:8081, svc-report), the Shell the attacker drives from its foothold, the Sudo Command (the find that svc-report runs as root via sudo) and the Root Cron Job. Data stores: the credentials file report.conf, the sudo policy (sudoers.d), the setuid binary backup-tool and the maintenance script rotate.sh. Each flow is labelled with what it carries and corresponds to something observed in Lab 1.

Trust boundaries: B1 – Network, the VM's network edge, crossed by the HTTP requests and responses between the Remote Client and report.service; B2 – Privilege (Root), which separates what runs as svc-report from what runs as, or is controlled by, root (sudo policy, Sudo Command, backup-tool, rotate.sh and the Root

Cron Job); B3 – Service account, which encloses the credentials file that only the service account should read, but which is world-readable.

Exercise 3a — The boundary expected to be attacked most

The trust boundary I expect to be attacked most is B2, because the attacker starts as svc-report and its goal is root, so every escalation must cross it. It is also the boundary with the most crossings, offering three possible paths to that goal: sudo find, the setuid backup-tool and writing to rotate.sh. B3 is crossed without any exploitation, since report.conf is world-readable, and B1 would only be the attacker's first step into the machine.

Part C — STRIDE, element by element

Exercise 4 — STRIDE table

The table applies the six categories (S – Spoofing, T – Tampering, R – Repudiation, I – Information Disclosure, D – Denial of Service, E – Elevation of Privilege) to every element of the DFD, with each flow on its own row. Following the applicability table in the handout, cells marked "—" are categories that do not apply to that element type (an external entity only admits S and R; a data store T, R, I and D; a flow T, I and D); cells marked "n/a" are applicable categories where, for the stated reason, no threat was identified. The three most serious cells (Exercise 4a) are highlighted in red.

Element	S	T	R	I	D	E
Remote Client (external entity, assumption)	Someone can impersonate the client, since report.service does not require authentication.	—	The service does not log what the client does, so the client can deny its actions.	—	—	—
Local User (student) (external entity)	Whoever steals the student password can log in over SSH (PasswordAuthentication yes) and gets full sudo.	—	n/a - The owner has no need to deny what they do on their	—	—	—

			own machine.			
report.service (:8081) (process)	No authentication: any client can pass itself off as a legitimate client.	n/a - the service does not write anything.	It does not log the requests it receives.	The response exposes db_user.	The pending-connection backlog is only 5 (Send-Q), so it is easy to exhaust with requests.	An unauthenticated remote client that compromises the service gets code execution on the machine as svc-report, i.e. the foothold of the attacker modelled in Part A.
Shell (process)	n/a - The Shell is used by the attacker, so there is no interest in impersonating it.	n/a - The shell binary is owned by root, so svc-report cannot modify it.	The Shell does not log actions against the respective users.	Through the Shell we can read report.conf and rotate.sh.	n/a - The attacker has no interest in taking down the Shell, since it is its foothold.	A user can escalate privileges through the Shell by changing the code of rotate.sh. They could also try through backup-tool or sudo find, but the reach of these has not yet been measured.
Sudo Command (find as root) (process)	It accepts svc-report without a password.	n/a - only root can write to the /usr/bin/find binary.	It was not checked whether sudo logs the users who run the command.	With sudo, svc-report can run find as root and therefore discover files or directories it could not otherwise access, although the reach of this command has not yet been measured.	n/a - the attacker would have no interest in making this process unavailable, since it can use it to its advantage.	svc-report, an unprivileged user, can run this command as root via sudo without a password, but its reach has not yet been measured.

Root Cron Job (process)	n/a - the root cron job is started by the system itself and takes no requests from other processes, so there is no identity the attacker could spoof.	Its behaviour can be changed by non-root users: it is enough to change rotate.sh.	We observed nothing that logs the root cron job's actions.	By changing rotate.sh, the root cron job, running it as root, can reveal the contents of files only root can read.	By changing rotate.sh, the root cron job stops doing its maintenance, since it executes that file as root.	By changing rotate.sh, the attacker gets its own code executed as root by the root cron job every 10 minutes.
report.conf (data store)	—	Only root can write this file, but the attacker can change it indirectly: it modifies rotate.sh so that the root cron job, running it as root, changes this file.	We did not check whether access to this file is logged, so actions may leave no trace.	All users can read the credentials in this file, so confidentiality is lost.	Only root can remove this file, but it is enough to change rotate.sh so that the root cron job deletes it; report.service would then no longer be able to respond.	—
Sudo policy (sudoers.d) (data store)	—	If rotate.sh is changed so that the root cron job, running it as root, modifies the sudo policy, the attacker can grant itself root privileges. Changing the policy gives persistent root, even after rotate.sh is fixed.	We did not check whether access to this file is logged, so actions may leave no trace.	If rotate.sh is changed so that the root cron job, running it as root, exposes the sudo policy, the attacker learns what privileges each user has.	If rotate.sh is changed so that the root cron job, running it as root, deletes this file, the local user loses full control of the machine.	—

backup-tool (data store)	—	Only root can write this file, but the attacker can change it indirectly: it modifies rotate.sh so that the root cron job, running it as root, changes this file.	We did not check whether access to this file is logged, so actions may leave no trace.	n/a - Any user can read this file, but it contains no secret.	Only root can remove this file, but it is enough to change rotate.sh so that the root cron job deletes it; backup-tool would then be unavailable.	—
rotate.sh (data store)	—	rotate.sh can be modified by every user. Since the root cron job executes it as root, a user can change the code to suit their goal.	We did not check whether changes to this file are logged, so actions may leave no trace.	n/a - Any user can read this file, but it contains no secret.	To remove this file it is enough to use cron, which runs it as root: the file is changed so that it deletes itself, and the maintenance it performs is lost.	—
Remote Client → report.service (HTTP request) (flow)	—	It can be altered in transit, for example in a man-in-the-middle attack, because the traffic is plain HTTP, without TLS.	—	n/a - the request carries nothing sensitive.	Flooding the service with requests stops it from answering legitimate clients.	—
report.service → Remote Client (HTTP response, db_user) (flow)	—	It can be altered in transit, for example in a man-in-the-middle attack, because the traffic is plain	—	A man-in-the-middle attacker can read the database user (db_user), because the traffic	Flooding the service with requests stops it from answering legitimate clients.	—

		HTTP, without TLS.		is plain HTTP, without TLS.		
Local User → report.service (administers) (flow)	—	n/a - the traffic does not leave the machine, so nobody on the network can alter or read it.	—	n/a - the traffic does not leave the machine, so nobody on the network can alter or read it.	If report.service is being flooded with requests, it cannot respond to the local user.	—
report.conf → report.service (reads) (flow)	—	Only root can write report.conf, but by changing rotate.sh so that the root cron job modifies it, report.service ends up reading wrong credentials.	—	All users can read the credentials in this file, so confidentiality is lost.	Only root can remove this file, but it is enough to change rotate.sh so that the root cron job deletes it; report.service would then be unable to respond, having no report.conf.	—
report.conf → Shell (world-readable) (flow)	—	n/a - the attacker has no reason to alter credentials it is reading itself.	—	All users can read the credentials in this file, so confidentiality is lost.	n/a - this flow is useful to the attacker for obtaining the credentials, so it has no reason to cut it.	—
Shell → Sudo policy (queries) (flow)	—	n/a - the traffic does not leave the machine, so nobody on the network can alter or read it.	—	n/a - it only reveals to svc-report its own rules, which it is already entitled to know.	n/a - this flow is useful to the attacker for learning its privileges, so it has no reason to cut it.	—

Shell → Sudo Command (sudo find) (flow)	—	n/a - the traffic does not leave the machine, so nobody on the network can alter or read it.	—	Through this flow the attacker can find files it would not otherwise have access to, since sudo find runs as root without a password. However, the reach of sudo find has not yet been measured.	n/a - this flow is useful to the attacker, which would lose access to files it could not otherwise find.	—
Shell → backup-tool (invokes) (flow)	—	n/a - the flow is local, so nobody on the network can read or alter it.	—	backup-tool runs as root and copies files, so whoever invokes it may end up reading files they could not otherwise access. However, the reach of this command has not yet been measured.	n/a - this flow is useful to the attacker, which would lose a way to escalate privileges and to copy any files it wanted.	—
Shell → rotate.sh (writes) (flow)	—	n/a - the flow is local, so nobody on the network can read or alter it.	—	n/a - what goes from the shell into the file is the code the attacker wants to run, which is no secret.	n/a - this flow is useful to the attacker, which would lose a way to escalate privileges and to run its code as root.	—

rotate.sh → Root Cron Job (reads and executes) (flow)	—	If the code of rotate.sh is changed, the root cron job executes it as root and the maintenance code becomes malicious code.	—	n/a - the flow is local, so nobody on the network can read it.	Cutting this flow leaves the machine without the rotate.sh maintenance: it is enough to change rotate.sh so that it deletes itself when the root cron job runs it.	—

Table 2 – STRIDE table of the VM (highlighted: the three most serious threats).

Exercise 4a — The three most serious cells

For the Root Cron Job element (process), the category is Elevation of Privilege: any user can escalate their privileges, since any user can modify rotate.sh, which cron executes as root. An attacker with access to the machine can change the code of this file, which cron will run as root, giving it full access to the machine. With the same method it can modify sudoers.d so that its own user has full control of the machine, equivalent to root: even if the rotate.sh problem is later fixed, the attacker keeps full access.

For the Shell element (process), the category is again Elevation of Privilege, since any unprivileged user can run /usr/local/bin/backup-tool, a cp with setuid root. The same applies to /usr/bin/find, which svc-report can run as root through sudo without a password. An attacker with access to the machine could therefore reach files it should not, although the real reach of these commands has not yet been measured.

For the report.conf element (data store), the category is Information Disclosure, since svc-report, like any other user, can read the credentials in this file. An attacker with access to any account can use these credentials on other systems.

Exercise 4b — A threat found only by working through the grid

A threat I would have missed without working through the grid systematically is Denial of Service on report.service (process). The service answers any client without authentication, so an attacker can flood it with requests until it stops answering legitimate ones. Indeed, in Lab 1, ss -tlnp showed a pending-connection backlog of

only 5 (Send-Q) on port 8081, against 4096 on the other ports. In Lab 1 I only saw this service as a source of information, because it returns db_user. It was the “D” question of the grid that made me think about its availability.

Part D — An attack tree

Exercise 5 — Goal G1: obtain root on the VM

Chosen goal: G1 – obtain root on the VM, starting from the unprivileged foothold of Part A (code running as svc-report). As the handout states that the root account is locked (* in the password field of /etc/shadow), reading /etc/shadow is not a route to root; every leaf of the tree therefore runs code as root or writes a file that root acts on.

Difficulty convention: at an AND node, the parent takes the difficulty of its hardest child; if two or more children share that maximum difficulty, it goes up one level (low → medium → high), high being the maximum. At an OR node, the parent takes the difficulty of its easiest child. “Deferred” leaves are left out of the cheapest-path calculation, because their difficulty is unknown.

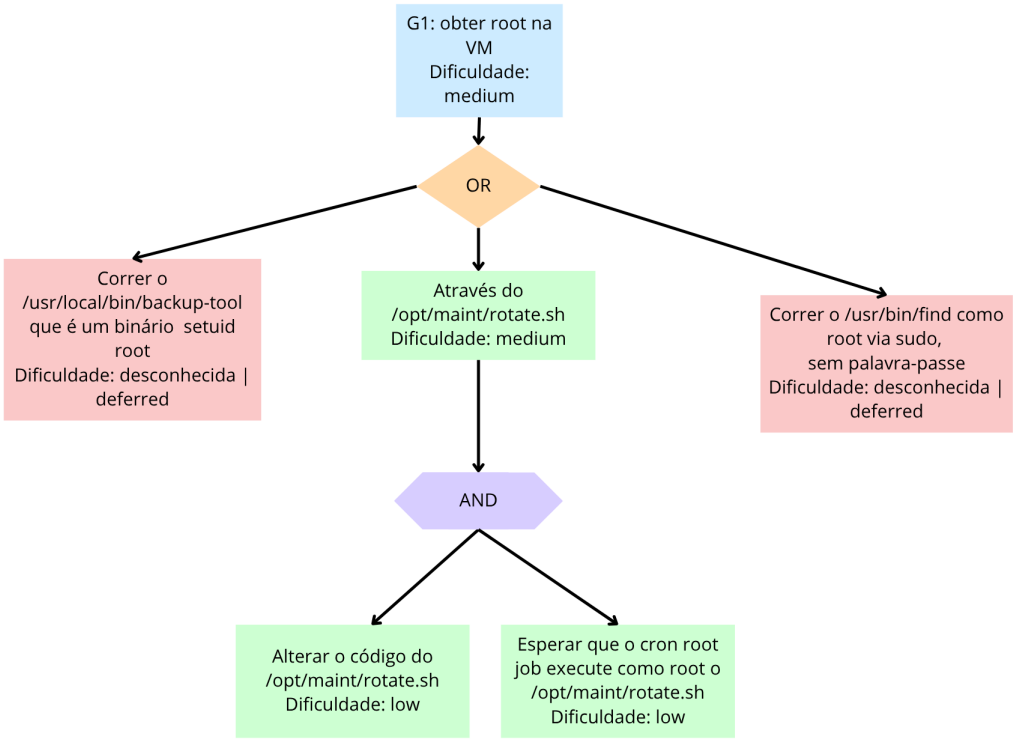


Figure 19 – Attack tree for G1 (obtain root on the VM), with the difficulty of each node. Labels in Portuguese: "obter root na VM" = obtain root on the VM; "Através do /opt/maint/rotate.sh" = through /opt/maint/rotate.sh; "Dificuldade: desconhecida | deferred" = difficulty: unknown | deferred.

Exercise 5a — Cheapest attack set

The leaves "Modify the code of /opt/maint/rotate.sh" and "Wait for the root cron job to run /opt/maint/rotate.sh as root" are both low and sit under an AND node; by the convention, the parent is medium. The OR node has two leaves and one AND node, but the leaves "Run /usr/bin/find as root via sudo, without a password" and "Run /usr/local/bin/backup-tool, a setuid-root binary" are marked "deferred", so they do not count. The OR node (and the G1 root) therefore takes the difficulty of its easiest child, the AND node "Through /opt/maint/rotate.sh": medium.

Cheapest attack set, in order:

1. Modify the code of /opt/maint/rotate.sh (world-writable) — low;
2. Wait for the root cron job (/etc/cron.d/ssc-maint, */10 * * * *) to run it as root, at most 10 minutes — low.

This is the cheapest path, since it needs no exploit, no tools that are not already on the machine, and no privileges beyond those of svc-report.

Exercise 5b — Deferred leaves

The attack tree has two deferred leaves: "Run /usr/local/bin/backup-tool, a setuid-root binary" and "Run /usr/bin/find as root via sudo, without a password". However, the cheapest path does not depend on them, because a complete path to root already exists using only things we understand: modify rotate.sh and wait for cron to run it as root.

After two labs, the most serious risk on this machine is already legible to us: there is a path to root privileges with no exploit, no extra tools and no privileges beyond those of svc-report. What we cannot yet evaluate are only the alternatives that depend on mechanisms taught later.

Part E — Prioritise, and measure

Exercise 6 — Ranked list of weaknesses

The list below ranks the machine's weaknesses from most to least urgent, based on the STRIDE table (Part C) and the attack tree (Part D).

#	Weakness	STRIDE	Remediation	Justification of rank
1	rotate.sh is world-writable and executed as root by cron	Elevation of Privilege	Remove write permission for group and others on rotate.sh, leaving write access to root only.	Modifying this file gives root and allows persistent access through sudoers.d, so it ranks above the second.
2	report.conf is readable by all users	Information Disclosure	Remove read access for others on report.conf and grant read access only to the svc-report group (owner root).	It ranks second because the credentials can be used outside the system to access the database. However, unlike rotate.sh, it does not give root.
3	backup-tool: any user can run this command as root	Elevation of Privilege	Remove the setuid bit from backup-tool, so that it runs with the identity of whoever executes it.	Although it runs as root, it ranks third because the reach of this command has not yet been measured.
4	sudo find: svc-report can run this command as root without a password	Elevation of Privilege	Remove the sudo rule that lets svc-report run find as root, since the service does not need it.	Although it runs as root, it ranks fourth because its reach has not yet been measured and it applies only to svc-report, not to all users.
5	The HTTP response of report.service exposes the database user	Information Disclosure	Stop including db_user in the HTTP response of report.service.	It ranks fifth because it only reveals the user name, not the password.
6	report.service can be flooded with requests and stop answering legitimate remote clients	Denial of Service	Make report.service listen only on the loopback interface (127.0.0.1).	It ranks sixth because the attacker has no interest in taking this service down, since it needs it for its foothold.
7	The student account accepts SSH password login (port 22 on 0.0.0.0, PasswordAuthentication yes), and that account has full sudo and belongs to the docker group.	Spoofing	Disable SSH password login (PasswordAuthentication no) and use public-key authentication.	It gives root to whoever obtains the student password, but it does not start from the position of the modelled attacker (svc-report) and needs an extra step, obtaining the password, so it ranks below the items svc-report can already exploit.

Table 3 – Weaknesses of the VM ranked by urgency of remediation.

Exercise 6a — Defending the top item against the second

rotate.sh ranks first because any user can modify it and the root cron job runs it as root, which allows escalation to root and full control of the machine, including persistently through the sudo policy. The report.conf credentials do not give this kind of access: they give access to the database, not to the machine. Moreover, the

damage of the first includes that of the second, since an attacker with root can also read report.conf, so fixing only report.conf would leave the path to the credentials open. Finally, fixing the permissions of report.conf does not stop the modelled attacker, who keeps reading it as svc-report, whereas fixing rotate.sh directly closes the cheapest path found in the attack tree.

Exercise 6b — An attack surface metric

The metric I propose is the number of privileged paths and exposed secrets that an unprivileged user can reach on the VM. It is the sum of five categories: files executed by root that any user can write, setuid-root files that any user can execute, credentials readable by any user, passwordless sudo rules granted to non-administrative accounts, and network services bound to 0.0.0.0 that answer without authentication.

For the count to be reproducible, each item counts as 1. Setuid binaries that are part of Ubuntu, such as passwd or sudo, are excluded, and only network services that answer without asking for credentials are included, so SSH does not count. Scripts executed by root are counted by checking with ls -l the permissions of the scripts referenced in /etc/cron.d/ that run as root, counting those that grant write access to group or others. Setuid binaries are obtained with find / -perm -4000 -type f, counting only those that are not standard Ubuntu binaries, such as backup-tool in /usr/local/bin, which Lab 1 distinguished from the 13 standard binaries of the image. Credentials are counted by looking for configuration files with passwords or keys that are readable by others (o+r). Sudo rules are obtained with sudo -l -U for each account that is not in the sudo group, counting the NOPASSWD entries. Finally, network services are counted from the ports bound to 0.0.0.0 or [::] in ss -tlnp whose service answers curl without credentials.

Category	Current	After fixing rotate.sh
World-writable and executed as root by cron	1 (rotate.sh)	0
Non-standard setuid-root binaries executable by any user	1 (backup-tool)	1
World-readable credentials	1 (report.conf)	1
Passwordless sudo rules granted to non-administrative accounts	1 (sudo find)	1

Network services bound to 0.0.0.0 that answer without authentication	1 (report.service)	1
Total	5	4

Table 4 – Value of the metric before and after the top-ranked remediation.

The current value for the VM is 5. With the top-ranked fix (removing write access for group and others on rotate.sh), it drops to 4. The metric is monotonic: every fix in the list of weaknesses lowers it, and a new writable file or a new exposed service raises it.

Exercise 6c — The policy line

The policy that the top-ranked fix assumes is: “On this machine, no unprivileged account, including svc-report, may modify scripts, binaries or configuration that root executes or relies on.” Besides removing write permission for group and others on rotate.sh, another mechanism that enforces the same line is removing the cron job that runs rotate.sh as root (/etc/cron.d/ssc-maint).

The two mechanisms are not equivalent. Removing write permission, leaving write access to root only, lets the maintenance keep running, which does not happen if the job is removed. On the other hand, removing the job guarantees that nothing is executed as root along that path, even if someone later gives write access to the file again by mistake, which the permission change alone does not guarantee. Because the two mechanisms differ, it is the policy that defines what is really forbidden, not just a permission change.

Lab 2 reflection

Doing the threat modelling first lets the defender think like the attacker and see which paths the attacker could take to reach a given goal, so the defender can decide how to guard against certain attacks. A model built after an attack tends to justify the path the attacker already used and to ignore the others, whereas modelling first forces all of them to be considered. In this lab, for example, I found three paths to root without exploiting any of them and chose to fix the cheapest one first, rotate.sh.

The DFD shows where attacks can happen through the trust boundaries. It was while drawing them that I realised the report.conf credential crosses boundary B3 without any exploitation. STRIDE allows the threat types to be assessed

systematically and omissions to be avoided, although it does not prove the model complete: it was the grid that led me to the Denial of Service on report.service, which the tree does not show because it is focused on obtaining root. The attack tree, in turn, shows which path is cheapest. While the STRIDE table gives similar weight to several threats, the tree showed that rotate.sh is the one the attacker would use first. Each instrument reveals something the others hide, so they are complementary rather than redundant.